**Foundations of Artificial Intelligence**

# AGENTIC DEBATE ARENA WITH MCTS

**A PROJECT REPORT**

***Submitted by***

**2116240701017 – S ADHITHYA**

**2116240701019 – ADITHYA S**




**October, 2026**

# BONAFIDE CERTIFICATE

Certified that this project report “AGENTIC DEBATE ARENA WITH MCTS” is the bonafide work of “S ADHITHYA (2116240701017) and ADITHYA S (2116240701019) who carried out the project work under my supervision.

**SIGNATURE OF THE FACULTY INCHARGE**

**Submitted for the Practical Examination held on _____**

**SIGNATURE OF THE INTERNAL EXAMINER**

# TABLE OF CONTENTS


| CHAPTER NO. | TITLE | PAGE NO. |
|---|---|---|
| ABSTRACT | | 1 |
| 1. INTRODUCTION | | 2 |
| | 1.1 BACKGROUND | 2 |
| | 1.2 PROBLEM STATEMENT | 2 |
| | 1.3 OBJECTIVES | 3 |
| | 1.4 SCOPE OF THE PROJECT | 3 |
| 2. LITERATURE REVIEW | | 4 |
| 3. PROPOSED SYSTEM | | 5 |
| | 3.1 ARCHITECTURE DIAGRAM | 5 |
| | 3.2 TECHNOLOGY STACK | 6 |
| | 3.3 WORKING PRINCIPLE | 6 |
| | 3.4 DESIGN PRINCIPLES | 6 |
| 4. MODULES DESCRIPTION | | 8 |
| | 4.1 MODULE 1: FACT BASE GENERATION AND DEBATE FRAMING | 8 |
| | 4.2 MODULE 2: KNOWLEDGE BASE (TELL/ASK) | 8 |
| | 4.3 MODULE 3: SEMANTIC FACT RETRIEVAL | 8 |
| | 4.4 MODULE 4: MONTE CARLO TREE SEARCH | 9 |
| | 4.5 MODULE 5: PROPOSITIONAL RESOLUTION | 10 |
| | 4.6 MODULE 6: AGENT AND TOOL LAYER | 11 |
| | 4.7 MODULE 7: BACKEND API AND SUMMARY GENERATION | 11 |
| | 4.8 MODULE 8: WEB INTERFACE | 12 |
| 5. IMPLEMENTATION AND RESULTS | | 13 |
| | 5.1 EXPERIMENTAL SETUP | 13 |
| | 5.2 CASE STUDY: A REPRESENTATIVE DEBATE | 13 |
| | 5.3 USER INTERFACE WALKTHROUGH | 14 |
| | 5.4 TESTING | 18 |
| | 5.5 DISCUSSION | 19 |
| 6. CONCLUSION AND FUTURE WORK | | 20 |
| 7. REFERENCES | | 21 |

# ABSTRACT

Debate is a demanding test of reasoning. A speaker must decide which point to press, remain consistent with earlier statements and answer an opponent whose arguments are not known in advance. Large language models (LLMs) write fluent arguments, but they offer no guarantee of consistency from one turn to the next, and their choice of strategy cannot be inspected.

This project presents the Agentic Debate Arena, a neuro-symbolic system in which two agents argue a user-chosen topic. On each turn a retriever selects evidence, a language model proposes candidate arguments, Monte Carlo Tree Search (MCTS) with the UCB1 rule chooses one through resolution-based simulations, and propositional resolution checks every claim. The backend uses Python and FastAPI, and the interface uses React.

The work covers Units I to IV of the Foundations of AI syllabus and is supported by 223 automated tests. At the end of a debate, the system names a winner from the contradictions proved by resolution and offers tactical coaching.

# 1. INTRODUCTION

## 1.1 BACKGROUND

Large language models can produce coherent text from a short prompt and adapt to new tasks from a few examples (Brown et al., 2020). They also state fluent falsehoods, a behaviour usually described as hallucination (Ji et al., 2023). A model that writes one side of a debate has no internal check that its third argument agrees with its first, and nothing in its output explains why one rebuttal was preferred to another.

Classical AI offers tools that are weak in language but strong in exactly these respects. A knowledge-based agent answers questions through the TELL and ASK operations (Russell and Norvig, 2020), resolution can prove a set of clauses inconsistent (Robinson, 1965), and Monte Carlo Tree Search balances untried moves against promising ones in a way that can be fully recorded (Coulom, 2007; Kocsis and Szepesvári, 2006). Neuro-symbolic systems combine the two traditions (Garcez and Lamb, 2023). A debate suits this approach because language quality matters, yet the outcome should depend on consistency and strategy. The arena therefore lets the LLM write while the symbolic modules decide.

## 1.2 PROBLEM STATEMENT

Automated debate systems based on language models alone face four difficulties:

- **Unverifiable consistency:** a model may contradict an earlier claim without any signal, and only a human reader can detect it.
- **Opaque strategy:** the choice of the next argument is hidden inside the model and cannot be audited or replayed.
- **Ungrounded facts:** claims are not tied to a fixed fact set, which leaves room for invented evidence.

- **Dependence on one provider:** rate limits and outages in a single API can stop a debate halfway through.

A debate system is therefore needed in which strategy and logical checking are carried out by transparent algorithms, the language model only proposes and phrases arguments, and the pipeline survives provider failures.

## 1.3 OBJECTIVES

The specific objectives of this project are:

1. To frame each topic as two positions and generate, once, a fact base of facts stored as propositional clauses.
2. To implement a TELL/ASK knowledge base for each debater (Unit I).
3. To retrieve the most relevant fact by cosine similarity and to select rebuttals with MCTS using UCB1 (Units II and III).
4. To detect contradictions by propositional resolution refutation (Unit IV).
5. To confine the LLM to proposing arguments, to support several providers with fallback, and to show the traces and verdict in a web interface.

## 1.4 SCOPE OF THE PROJECT

The system supports debates between two AI agents on any topic typed by the user, over 6, 10, 20 or 30 turns. Every topic is framed as a resolution with an explicit position for each side, and no topic depends on hand-written arguments. The logic is propositional, so first-order reasoning lies outside the scope of the work.

The system is a teaching and demonstration platform. It does not verify the facts in its fact base or replace a human adjudicator, and sessions are held in memory only. The winner is decided by the contradictions proved by resolution, which measure logical standing, not rhetoric.

# 2. LITERATURE REVIEW

### *Knowledge-based agents and resolution*

A knowledge-based agent stores sentences in a knowledge base and interacts with it through TELL, which adds a sentence, and ASK, which queries what follows (Russell and Norvig, 2020). Resolution is refutation-complete (Robinson, 1965): if a set of clauses is unsatisfiable, the empty clause will eventually be derived, and every step can be recorded as a checkable proof.

### *Search and retrieval*

Monte Carlo Tree Search grows a game tree selectively, guided by simulated outcomes (Coulom, 2007). Kocsis and Szepesvári (2006) combined it with the UCB1 bandit rule of Auer et al. (2002), giving the UCT algorithm. For retrieval, texts are represented as vectors and ranked by cosine similarity (Manning, Raghavan and Schütze, 2008), which captures meaning when the vectors come from neural encoders.

### *Language-model agents and debate*

The ReAct approach interleaves reasoning with external tool calls (Yao et al., 2023), although the model still decides which tool to use. Irving, Christiano and Amodei (2018) proposed that two agents argue before a human judge, and Du et al. (2023) found that multi-agent debate improved factual accuracy. In both, the debaters are language models, and little attention has been paid to enforcing logical consistency formally.

### *Gap identified*

Garcez and Lamb (2023) argue that learning should be combined with explicit reasoning, which supports the division of labour adopted here. To the best of our knowledge, few student-scale systems combine MCTS-based move selection, resolution-based consistency checking, grounded retrieval and interchangeable LLM providers, and the present work addresses this gap.

# 3. PROPOSED SYSTEM

The proposed system is a debate arena in which two agents argue opposite sides of a topic. Its design rests on one rule: the language model may propose and phrase arguments, but it may not decide. Strategy belongs to MCTS, contradiction detection to propositional resolution and the choice of evidence to a cosine-similarity retriever, so every decision can be traced to an algorithm and its numbers.

## 3.1 ARCHITECTURE DIAGRAM

The architecture is shown in Figure 3.1. The React interface talks to a FastAPI service, which hands each request to an agent. The agent calls tools through a single executor, which in turn calls the symbolic core. The language models sit at the bottom and supply language only, with the fact base as the fallback.

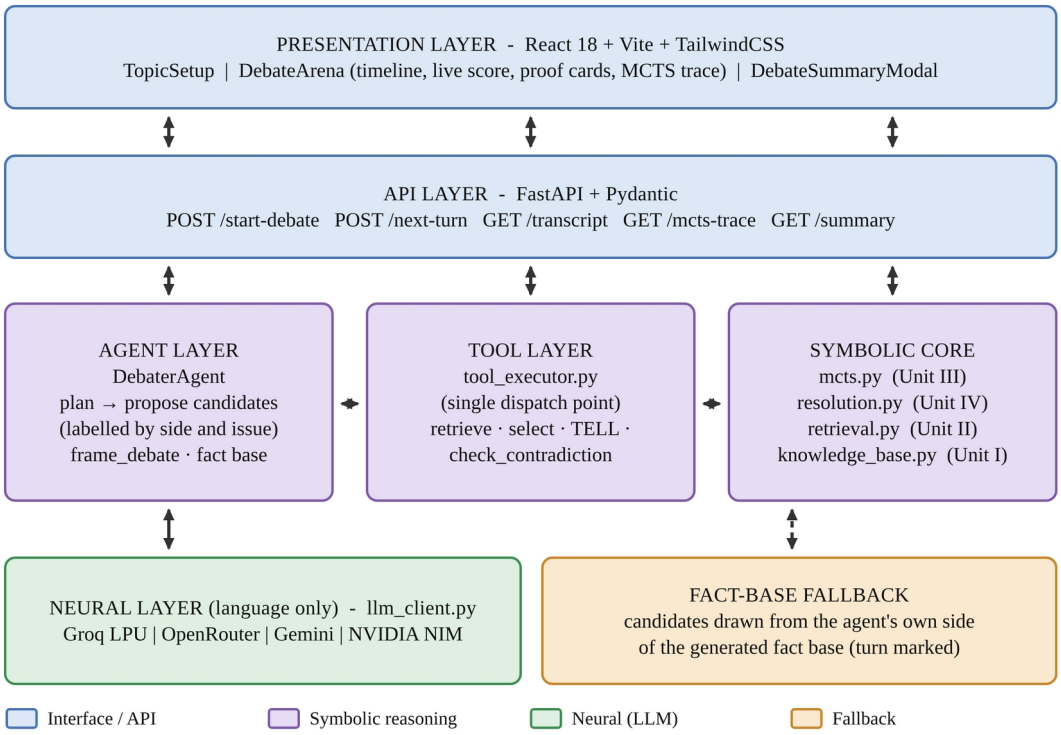



**Figure 3.1: System architecture of the Agentic Debate Arena**

## 3.2 TECHNOLOGY STACK

The backend is written in Python 3.10+ with FastAPI and Pydantic for the web layer and NumPy for vectorised cosine similarity. Language models are reached through one client, using the Google GenAI SDK or the OpenAI-compatible SDK for Groq, NVIDIA NIM and OpenRouter. The interface is a React 18 single-page application built with Vite and TailwindCSS, tested with pytest.

## 3.3 WORKING PRINCIPLE

A debate runs in two phases. In the set-up phase, a language model frames the topic as two explicit positions and then generates about twenty facts, each stored as a sentence, a propositional form and a side label. The sentences are indexed once, and each fact joins the knowledge base of its side if that keeps it consistent.

In the debate phase, each call to the next-turn endpoint advances the debate by one turn, and the agents alternate. Each turn has five steps: the agent retrieves the relevant evidence, the LLM proposes about four labelled candidates, MCTS selects one by simulating its logical consequences, the clause is told to both knowledge bases, and resolution checks for self-contradiction and refutation. Finally, the system names a winner and writes a debrief.

## 3.4 DESIGN PRINCIPLES

Four constraints govern the design: the language model never decides whether two claims contradict, never chooses which argument is spoken and never adds facts during the debate, and a model failure mid-debate still leaves the real algorithms running. Table 3.2 lists who is responsible for each one.

**Table 3.2: Division of labour between symbolic and neural components**

| Decision | Component responsible | Role of the LLM |
|---|---|---|
| Which fact is relevant | Cosine-similarity retrieval (retrieval.py) | None |
| Which argument is spoken | MCTS with resolution rollouts (mcts.py) | None |
| Whether a contradiction exists | Resolution refutation (resolution.py) | None |
| Which facts exist in the debate | One-time generation at set-up (fact_generation.py) | Generates the fact base once |
| How a claim is worded | Candidate generation (debater_agent.py) | Proposes and words the candidates |

The project also maps directly onto the course syllabus: knowledge-based agents (Unit I) correspond to the TELL/ASK knowledge base, informed search (Unit II) to cosine-similarity retrieval, adversarial search (Unit III) to MCTS and propositional logic (Unit IV) to resolution.

# 4. MODULES DESCRIPTION

The system is divided into eight modules, each with a single responsibility. They communicate through the tool executor and the shared knowledge-base objects.

## 4.1 MODULE 1: FACT BASE GENERATION AND DEBATE FRAMING

This module prepares the debate. A language model first frames the topic, so that a question such as “Gaming laptop or business laptop?” becomes a resolution which the Proposer defends and the Opponent rejects. It then returns about twenty facts, each with a sentence, a propositional form such as A → B, ¬A or A, and a side label; atom names must name their subject, for example GamingLaptopWeightHeavy. The parser tolerates markdown fences and reasoning tags. Generation tries Groq first in JSON mode and then the other providers, and if none responds the debate is not started, so no topic falls back to scripted content.

## 4.2 MODULE 2: KNOWLEDGE BASE (TELL/ASK)

Each agent owns a KnowledgeBase object that follows the TELL and ASK architecture (Russell and Norvig, 2020). It stores the agent’s own clauses, the opponent’s claims and a readable history. Facts are told only if they keep the knowledge base satisfiable, and a claim proved self-contradictory is recorded and then retracted, so the agent’s commitments stay consistent and refutable. The ASK methods pass these clause sets to the resolution engine to reveal self-contradiction and clashes.

## 4.3 MODULE 3: SEMANTIC FACT RETRIEVAL

Retrieval follows the idea of informed search in Unit II, with the similarity between the query and each stored fact acting as the heuristic. All facts are embedded once when the debate is created and stored in a FactIndex, and for each turn the cosine similarity with every row is computed in one vectorised operation:

$$sim(a, b) = (a \cdot b) / (\|a\| \times \|b\|)$$

The result lies between −1 and 1, and a zero vector is given a similarity of 0. The top-k facts are returned with their scores. The embedding function is injected: the system uses a local hashed bag-of-words encoding that splits atom names into words; the tests use fixed vectors.

### 4.4 MODULE 4: MONTE CARLO TREE SEARCH

The strategic core is mcts.py. Each child of the root is a candidate argument proposed by the LLM, and each simulation scores a candidate C with a resolution rollout

$$R(C) = refute - 1.5 \cdot self - 0.5 \cdot exposed + 0.4 \cdot new - 0.3 \cdot stale + \varepsilon$$

where refute marks a contradiction with a claim the opponent has made, self one with the agent's own knowledge base, exposed one that a sampled opponent reply would derive, new an issue not yet debated and stale an atom argued three times recently, while ε adds small bonuses for engaging the last claim and using fact-base atoms. Nodes are selected with the UCB1 rule:

$$UCB1 = W / N + C \times \sqrt{( \ln N_p / N )}, \quad C = \sqrt{2}$$

where W is the total value passed back through the node, N its visit count and N_p that of its parent. Unvisited nodes score infinity, so every candidate is tried once. As Figure 4.1 shows, selection descends by UCB1, expansion adds an untried child, simulation runs a resolution rollout and backpropagation updates the statistics. After 32 simulations per turn, the most visited child is chosen, since visit count is a more stable estimate than raw value.

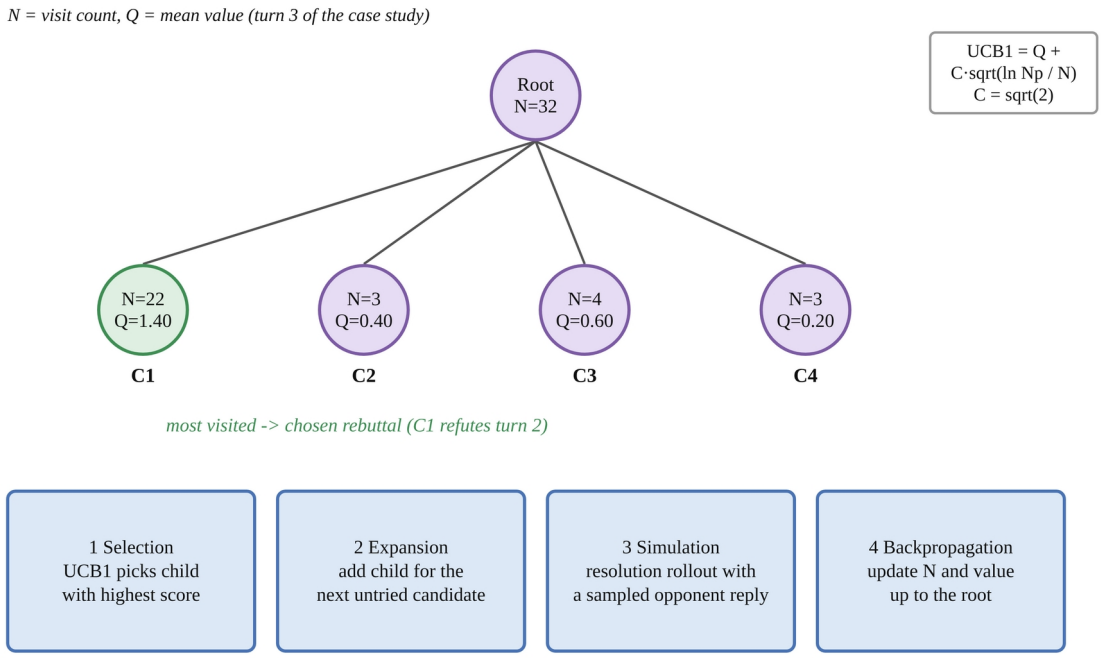



**Figure 4.1: Monte Carlo Tree Search over candidates (turn 3 of the case study)**

The interface lists the rejected candidates with their average scores and visit counts, so that the reader can see what the agent considered.

## 4.5 MODULE 5: PROPOSITIONAL RESOLUTION

Contradiction detection is the work of resolution.py. A literal is an atom or its negation, a clause is a set of literals standing for their disjunction, and an implication P → Q is stored as {¬P, Q}. The resolution rule takes two clauses with a complementary pair of literals and returns a clause made from the remainder:

$$(A \vee B),\ (\neg A \vee C) \vdash (B \vee C)$$

The function check_contradiction resolves every pair of clauses, discards tautologies and duplicates, and adds new resolvents until either the empty clause appears, in which case a contradiction is proved, or no new clause can be generated, in which case the set is satisfiable. Each step is recorded as RESOLVE {A ∨ B} + {¬A ∨ C} → {B ∨ C}. Unparseable forms are never asserted, and a refutation must contradict something the opponent actually said.

## 4.6 MODULE 6: AGENT AND TOOL LAYER

A single DebaterAgent class serves every provider. Each turn is a fixed sequence of calls through one dispatch function, the tool executor (Table 4.1), so the agent never reaches the algorithms directly.

**Table 4.1: Tools available to the debater agents**

| Tool | Algorithm | Output |
|---|---|---|
| retrieve_relevant_fact | Cosine similarity (retrieval.py) | Top-k facts with similarity scores |
| select_best_rebuttal | MCTS with resolution rollouts (mcts.py) | Chosen argument and rejected candidates with scores |
| generate_claim | Clause is TELLed to both knowledge bases | Updated knowledge bases and claim history |
| check_contradiction | Resolution (resolution.py) | Self and clash results with proof steps |

Before proposing candidates, the agent reads the recent exchange and the issues already argued, writes a short plan, and drafts arguments that either rebut the last claim or open a new issue; candidates that would help the other side are discarded. Each model retries once after a rate-limit error and then moves along its fallback chain. If every model fails, candidates are taken from the agent's own side of the fact base and the turn is marked as a fallback.

## 4.7 MODULE 7: BACKEND API AND SUMMARY GENERATION

The FastAPI application exposes five endpoints: start-debate, next-turn, transcript, mcts-trace and summary. Sessions are held in an in-memory dictionary, and each call to next-turn selects the speaker by turn parity, builds an ExecutorContext, runs the agent's turn and records the claim, the contradiction result and the MCTS trace.

The summary endpoint names the winner from the refutations and self-contradictions proved during the debate. It then asks a language model for a debrief grounded in the transcript and the proofs, and generates a counter-reply and an escape route for each side's strongest attack, recorded as CorneringPoints. If no provider is reachable, the summary is built from the recorded claims.

### 4.8 MODULE 8: WEB INTERFACE

The interface is a React single-page application. The landing page lets the user type a topic, select a model for each agent from four provider groups and choose 6, 10, 20 or 30 turns. During a debate, the header states both positions and a live score of proven refutations, and a vertical timeline shows Agent A's claims on the left and Agent B's on the right, each with its issue, propositional form and status badge. A refutation appears as a proof card linked by an arc to the turn it refutes, and each turn's MCTS candidates can be expanded. The verdict report offers the result, tactical coaching and the full arguments, and three themes (Brief, Ledger and Nocturne) are available.

# 5. IMPLEMENTATION AND RESULTS

## 5.1 EXPERIMENTAL SETUP

The system was run on a development machine with a single launcher script, run_app.sh, which starts the backend on port 8000 and the frontend on port 5173. API keys are read from a .env file. The main settings are listed in Table 5.1.

**Table 5.1: Experimental setup**

| Item | Details |
|---|---|
| Backend | Python 3.10+, FastAPI, Pydantic, NumPy |
| Frontend | React 18.3, Vite 5.2, TailwindCSS 3.4, lucide-react |
| MCTS simulations | 32 per turn (configurable in config.py) |
| Fact base size | 20 facts per debate |
| Debate length | 6, 10, 20 or 30 turns (default 20) |

## 5.2 CASE STUDY: A REPRESENTATIVE DEBATE

The topic “Gaming laptop or business laptop for students in college?” illustrates a session, with Agent A on GPT-OSS 120B, Agent B on DeepSeek V3.2 and ten turns. The resolution became “College students should buy a gaming laptop rather than a business laptop”, and the facts included “Gaming laptops are heavier than most business laptops”. Table 5.2 summarises turns 2 and 3.

**Table 5.2: Steps of turns 2 and 3**

| Turn | Step | Component | Outcome |
|---|---|---|---|
| 2 (B) | Propose and select | mcts.py | Weight argument chosen from three candidates |
| 2 (B) | Check self and clash | resolution.py | No contradiction found |
| 3 (A) | Propose and select | mcts.py | Backpack rebuttal chosen (score 1.40, 22 of 32 visits) |
| 3 (A) | Check self and clash | resolution.py | Refutes turn 2: resolution derives the empty clause |

Turn 3 shows how the modules interact. Agent B argued that heavy gaming laptops reduce portability, and its evidence stated that gaming laptops are heavy. When Agent A denied the consequence, resolution derived the empty clause from B's claim and evidence together, and MCTS had already preferred this candidate because its rollouts detected the refutation. Agent A won 2–0, and the proofs are reproducible for the same candidates.

## 5.3 USER INTERFACE WALKTHROUGH

The main screens are shown below. Figure 5.1 shows the landing page, where the topic and the models are chosen.

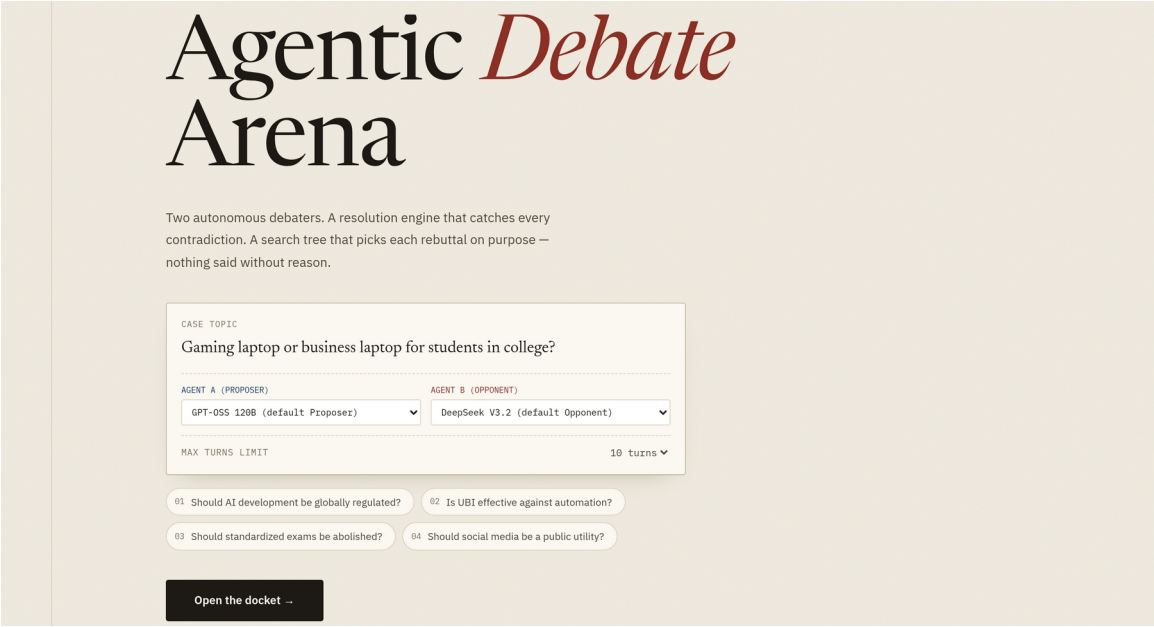



**Figure 5.1: Landing page with topic input, model selectors and suggested topics**

The debate is displayed as a timeline (Figure 5.2), with the two agents alternating on the left and right.

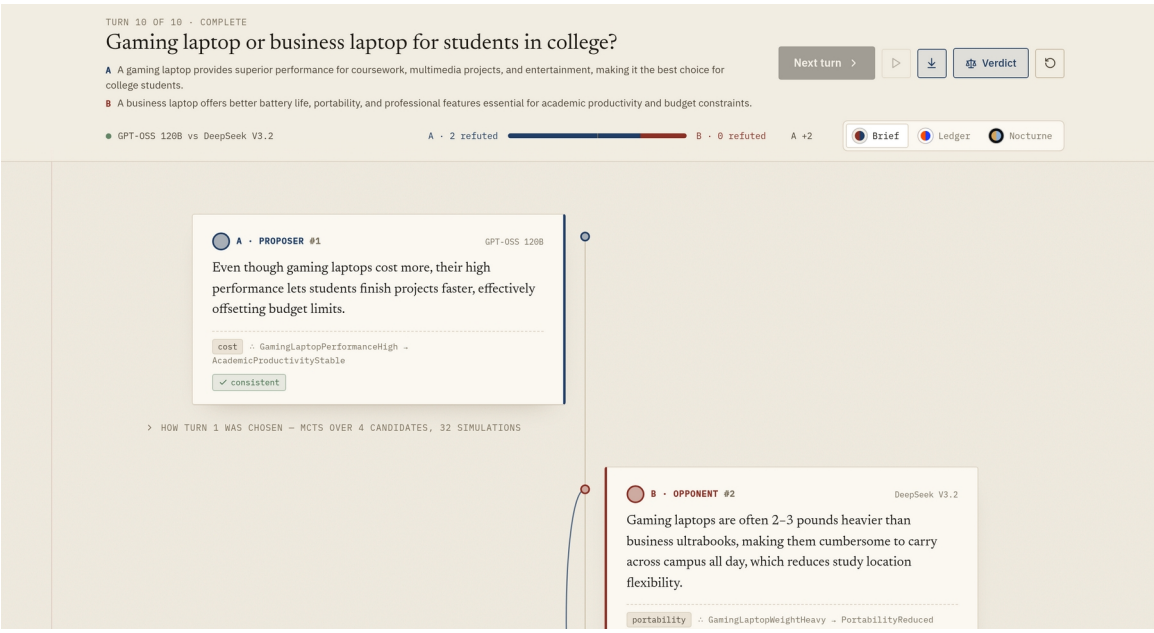



**Figure 5.2: Debate timeline with alternating claims from Agent A and Agent B**

Figure 5.3 shows the MCTS decision trace for one turn, listing the chosen and rejected rebuttals with their scores and visit counts.




**Figure 5.3: MCTS trace for turn 3 with the chosen and rejected candidates**

When resolution proves a refutation, the proof is displayed as in Figure 5.4.




**Figure 5.4: Refutation card with the resolution proof for turn 3**

The verdict report opens with the result (Figure 5.5), which names the winner and the refutations proved by each side.

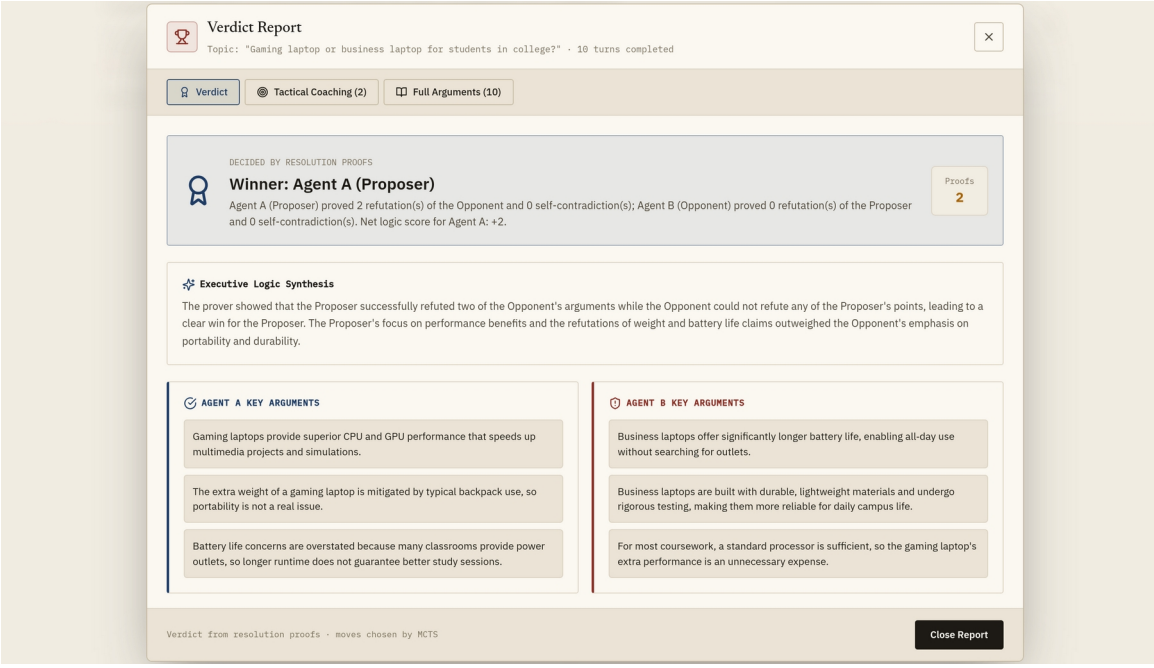



**Figure 5.5: Verdict report: winner decided by resolution**

## 5.4 TESTING

The backend is covered by twelve test files containing 223 unit and integration tests, grouped in Table 5.3. Every call to a language model is replaced by a deterministic fake, so the suite runs offline in under two seconds, and separate tests confirm that the fallback mode still runs the real MCTS and resolution code.

**Table 5.3: Test suite overview**

| Test file | Tests | Coverage |
|---|---|---|
| test_mcts.py | 52 | UCB1, selection, expansion, backpropagation, resolution rollouts, edge cases |
| test_resolution.py | 42 | Negation, resolve, tautologies, contradiction detection, parsing |
| test_tool_executor.py | 50 | Tool handlers, clash and refutation rules, error handling |
| test_retrieval.py | 28 | Cosine similarity, index building, top-k retrieval, edge cases |
| Remaining files | 51 | Knowledge base, routes, framing, end-to-end flow, claim cleaning, fallback |

Figure 5.6 shows the summary line of a complete pytest run.

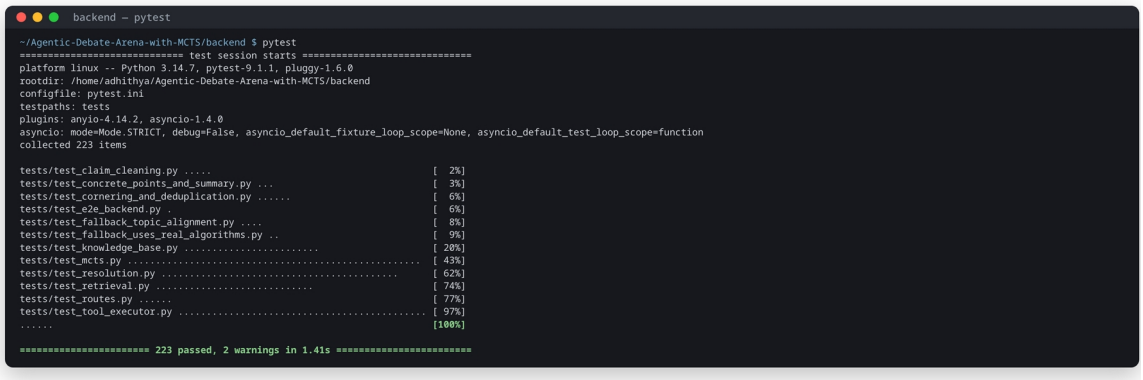



**Figure 5.6: Terminal output of the pytest run**

## 5.5 DISCUSSION

The main design goal was met: every strategic decision and every contradiction verdict can be traced to a numerical record, and the language model only proposes and phrases arguments. When a provider fails mid-debate, the agent falls back to its own side of the fact base, as happened once in the case study, and the logic is unaffected. The cost is modest. With 32 simulations over two to five candidates, MCTS and retrieval are inexpensive, and resolution, which is quadratic in the number of clauses in the worst case, completes quickly on knowledge bases of a few dozen clauses.

Several limitations should be acknowledged. The search tree has one level of candidates with sampled opponent replies, so it resembles bandit allocation more than a deep game tree. The logic is propositional, and a propositional form that does not match its sentence can hide a real conflict or create a false one. Turns depend on external providers whose rate limits and latency vary, sessions are lost when the server restarts, and the verdict has not been validated against human judgement or across many topics.

# 6. CONCLUSION AND FUTURE WORK

***Conclusion***

This project developed the Agentic Debate Arena, a neuro-symbolic system in which two agents debate any topic under the control of classical AI algorithms. Cosine-similarity retrieval selects the evidence, the language model proposes candidate arguments, MCTS with UCB1 selects one through resolution-based simulations and propositional resolution checks every claim. As each topic is framed into two positions and a refutation must target what the opponent actually said, the outcome reflects the logic of the exchange, and the interface exposes the proofs behind it.

The main contribution is architectural: a clear division of labour, in which the neural component proposes and the symbolic components decide, produces a system whose behaviour can be explained, tested and reproduced.

***Future work***

- Extend the rollouts from one sampled reply to multi-step exchanges between the two agents.
- Move to first-order logic so that quantified claims can be checked.
- Validate the proof-based verdict against human ratings of debates.
- Persist sessions in a database so that debates can be replayed, and allow a human to compete against the MCTS-guided opponent.

# 7. REFERENCES


Auer, P., Cesa-Bianchi, N. and Fischer, P. (2002) 'Finite-time Analysis of the Multiarmed Bandit Problem', Machine Learning, Vol.47, No.2-3, pp.235-256.

Brown, T.B., Mann, B., Ryder, N. et al. (2020) 'Language Models are Few-Shot Learners', Advances in Neural Information Processing Systems, Vol.33, pp.1877-1901.

Coulom, R. (2007) 'Efficient Selectivity and Backup Operators in Monte-Carlo Tree Search', Computers and Games (CG 2006), Lecture Notes in Computer Science, Vol.4630, pp.72-83.

Du, Y., Li, S., Torralba, A., Tenenbaum, J.B. and Mordatch, I. (2023) Improving Factuality and Reasoning in Language Models through Multiagent Debate. arXiv:2305.14325.

Garcez, A.d'A. and Lamb, L.C. (2023) 'Neurosymbolic AI: The 3rd Wave', Artificial Intelligence Review, Vol.56, pp.12387-12406.

Irving, G., Christiano, P. and Amodei, D. (2018) AI Safety via Debate. arXiv:1805.00899.

Ji, Z., Lee, N., Frieske, R. et al. (2023) 'Survey of Hallucination in Natural Language Generation', ACM Computing Surveys, Vol.55, No.12, Article 248.

Kocsis, L. and Szepesvári, C. (2006) 'Bandit Based Monte-Carlo Planning', Machine Learning: ECML 2006, Lecture Notes in Computer Science, Vol.4212, pp.282-293.

Manning, C.D., Raghavan, P. and Schütze, H. (2008) Introduction to Information Retrieval. Cambridge: Cambridge University Press.

Robinson, J.A. (1965) 'A Machine-Oriented Logic Based on the Resolution Principle', Journal of the ACM, Vol.12, No.1, pp.23-41.

Russell, S. and Norvig, P. (2020) Artificial Intelligence: A Modern Approach, 4th edn. Harlow: Pearson.

Yao, S., Zhao, J., Yu, D. et al. (2023) 'ReAct: Synergizing Reasoning and Acting in Language Models', International Conference on Learning Representations (ICLR 2023).